
UNIVERSITÉ PARIS NANTERRE

Master 1 MIAGE

Optimisation combinatoire

GraphBench Challenge

Refutation automatique de conjectures en théorie des graphes

Binôme : Seyed Amineddin MIRI et Marie BIBI
Enseignement : Optimisation combinatoire
Enseignant : François DELBOT
Année universitaire : 2025–2026
Date de rendu : 9 mai 2026
Dépôt GitHub : <https://github.com/Aminmiri82/projet-optimisation-combinatoire>

Conjectures réfutées : 100/100
Score final ($\sum_i c_i$) : 136,80
Temps total Partie 1 : 136,80 s
Temps moyen Partie 1 : 1,368 s
FunSearch (GPT-5,5, `candidate_006`) : 100/100
FunSearch (DeepSeek, `candidate_014`) : 100/100

Table des matières

1	Introduction et objectif	2
2	Heuristique simple	3
2.1	Principe de validation	3
2.2	Boucle de recherche	3
2.3	Évolution de <code>generate_initial_population</code>	4
2.4	Score manuel	5
3	Architecture FunSearch	6
3.1	Analyse du candidat GPT-5.5 <code>candidate_006</code>	6
3.2	Analyse du candidat DeepSeek <code>candidate_014</code>	7
4	Résultats expérimentaux	7
5	Discussion scientifique	7
5.1	Récapitulatif des questions du sujet	8
6	Dépendances et bibliothèques	8

1 Introduction et objectif

Le projet GraphBench demande de réfuter automatiquement des conjectures de théorie des graphes. Une conjecture compare deux invariants d'un graphe, par exemple

$$y(G) \leq f(x(G)) \quad \text{ou} \quad y(G) \geq f(x(G)).$$

Un contre-exemple est un graphe appartenant aux classes imposées par le benchmark, pour lequel l'inégalité est strictement violée. Dans le benchmark utilisé, les classes principales sont **connected**, **tree** et **claw_free**.

Le système doit donc charger les conjectures, générer des graphes admissibles, calculer les invariants requis, maximiser la violation, puis produire un graphe au format **graph6**. Notre solution se décompose en deux parties. La Partie 1 est une recherche heuristique conçue à la main. La Partie 2 implémente une boucle de type FunSearch où un modèle de langage propose des fonctions de score, qui sont ensuite évaluées automatiquement.

Le résultat principal est le suivant : la Partie 1 atteint 100/100 contre-exemples valides avec un coût total de 136.801705 secondes, soit 1.368017 seconde en moyenne. FunSearch atteint également 100/100 avec certains candidats, mais ne dépasse pas le temps de l'heuristique manuelle finale.

Partie 1

2 Heuristique simple

2.1 Principe de validation

La validation finale utilise uniquement la violation mathématique de la conjecture. Pour une conjecture $y \leq f(x)$, la violation est

$$\text{violation}(G) = y(G) - f(x(G)).$$

Pour une conjecture $y \geq f(x)$, elle est

$$\text{violation}(G) = f(x(G)) - y(G).$$

Un graphe est accepté seulement si la violation est strictement positive et si les contraintes de classe sont vérifiées après recalcul des invariants. Cette séparation est importante : le score heuristique peut guider la recherche, mais il ne peut pas valider un faux contre-exemple.

Module	Role
<code>loader.py</code>	Chargement du benchmark CSV
<code>conjecture.py</code>	Objet conjecture et calcul exact de la violation
<code>invariants.py</code>	Calcul des invariants avec <code>networkx</code> et <code>numpy</code>
<code>classes.py</code>	Tests <code>connected</code> , <code>tree</code> , <code>claw_free</code>
<code>generators.py</code>	Génération initiale et familles de graphes
<code>mutations.py</code>	Modifications locales de graphes
<code>repair.py</code>	Réparation après génération ou mutation
<code>search.py</code>	Sélection, mutation, évaluation et arrêt
<code>verify.py</code>	Vérification stricte des graphes <code>graph6</code>

TABLE 1 – Architecture de la Partie 1.

2.2 Boucle de recherche

La fonction principale conserve une population de graphes scores. Elle commence par une population initiale, sélectionne ensuite des parents prometteurs, applique une mutation locale ou injecte parfois un graphe frais, répare le graphe si nécessaire, puis calcule seulement les invariants utiles à la conjecture courante. Les candidats sont triés par score objectif, mais la condition d'arrêt reste la validation stricte.

Le cœur de l'évaluation est volontairement robuste aux scores générés automatiquement. Si un score lève une exception, retourne `None` ou produit une valeur non finie, l'objectif devient $-\infty$, mais le moteur continue.

Listing 1 – Evaluation d'un candidat dans le moteur de recherche.

```
def _evaluate(graph, conjecture, needed, scorer):
    invariants = compute_cached(graph, needed)
    try:
        raw_objective = scorer(graph, invariants, conjecture)
        objective = float(raw_objective) if raw_objective is not None else float("-inf")
    except Exception:
        objective = float("-inf")
    if not math.isfinite(objective):
        objective = float("-inf")
    return objective, violation_score(graph, invariants, conjecture), invariants
```

2.3 Évolution de `generate_initial_population`

L'amélioration la plus décisive de la Partie 1 n'a pas été le score seul, mais l'espace de graphes donné au moteur. Au départ, `generate_initial_population` ne faisait qu'échantillonner aléatoirement des graphes selon la classe demandée. Cette approche explore, mais elle manque les structures extrémales rares.

Listing 2 – Version v1 : population aléatoire réparée.

```
population = []
while len(population) < size and attempts < size * 80:
    n = rng.randint(min_order, max_order)
    graph = generate_graph(classes, rng, n)
    graph = repair(graph, classes, rng)
    if graph is not None and satisfies_classes(graph, classes):
        population.append(_normalize_nodes(graph))
```

En v2, nous avons ajouté une phase déterministe avant l'échantillonnage aléatoire. La population commence par des graphes simples et quelques familles ciblées. Ce changement transforme la recherche : au lieu d'attendre qu'une mutation aléatoire tombe sur une bonne forme, le moteur dispose immédiatement de contre-exemples proches.

Listing 3 – Version v2 et suivantes : graines structurelles avant le hasard.

```
for graph in seed_graphs(classes, min_order, max_order):
    graph = repair(graph, classes, rng)
    if graph is not None and satisfies_classes(graph, classes):
        population.append(_normalize_nodes(graph))
if len(population) >= size:
    return population
```

Les premières graines étaient des chemins, cycles, cliques et araignées. Les araignées ont été ajoutées car elles augmentent certains invariants de domination sans augmenter aussi vite le couplage. Elles sont construites à partir d'un centre et de plusieurs pattes de longueurs contrôlées.

Listing 4 – Araignées introduites en v2.

```
def _spider(lengths):
    graph = nx.Graph()
    graph.add_node(0)
    next_node = 1
    for length in lengths:
        previous = 0
        for _ in range(length):
            graph.add_edge(previous, next_node)
            previous = next_node
            next_node += 1
    return graph
```

En v3, la population initiale s'est élargie avec des doubles étoiles, des chaînes de cliques endommagées, des graphes connexes clairsemés, des triangles avec chemins attachés et des graphes clique–chemin–clique. Les doubles étoiles ciblent les cas où l'on veut beaucoup de feuilles autour de peu de sommets centraux. Les chaînes de cliques endommagées donnent une densité et des triangles élevés tout en modifiant le degré maximal ou la connectivité. Les graphes clique–chemin–clique sont utiles pour des conjectures où les grandeurs spectrales et les distances réagissent fortement à un goulot d'étranglement.

Listing 5 – Exemple de graine clique–chemin–clique.

```
def _clique_path_clique(left_size, right_size, path_length):
    left = list(range(left_size))
    path = list(range(left_size, left_size + path_length))
    right = list(range(left_size + path_length,
                      left_size + path_length + right_size))
    graph.add_edges_from((u, v) for i, u in enumerate(left)
                        for v in left[i + 1:])
    graph.add_edges_from((u, v) for i, u in enumerate(right)
                        for v in right[i + 1:])
    previous = left[0]
    for node in path:
        graph.add_edge(previous, node)
        previous = node
    graph.add_edge(previous, right[0])
```

La v4 a stabilisé les derniers échecs en ajoutant les chemins branchés et les queues triangulaires. Les chemins branchés gardent une structure d'arbre, mais créent localement des effets forts sur la domination indépendante, les feuilles et certains indices de Zagreb. Les queues triangulaires conservent une structure proche du sans-griffe tout en contrôlant les distances.

Version	Ajouts dans la population initiale	Motivation
v1	Génération aléatoire par classe : arbres, chemins, cycles, cliques, $G(n, p)$, graphes denses sans griffe.	Obtenir une base modulaire et vérifier le pipeline.
v2	Graines déterministes, chemins/cycles/cli-ques de tailles fixes, araignées.	Donner au moteur des graphes extrémaux simples dès le début.
v3	Doubles étoiles, cliques endommagées, graphes clairsemés, triangles avec chemins, clique–chemin–clique.	Cibler domination, densité, triangles, distances et effets spectraux.
v4	Chemins branchés et queues triangulaires.	Résoudre les derniers cas d'arbres, de graphes sans griffe et de distances spectrales.

TABLE 2 – Évolution de `generate_initial_population`.

2.4 Score manuel

Le score manuel part de la violation brute et ajoute de faibles termes de guidage normalisés. Le coefficient dominant reste la violation, afin de ne pas éloigner la recherche de l'objectif mathématique.

Listing 6 – Forme simplifiée du score manuel.

```
score = 100.0 * violation
score += 0.15 * y_value if conjecture.sign == "<=" else -0.15 * y_value
score += 0.12 * clamp(direction_from_derivative) * x_value
score += structural_guidance(G, invariants, conjecture)
```

Les termes structurels regardent la densité, la proportion de feuilles, les triangles et le diamètre. Ils favorisent les formes cohérentes avec l'invariant que l'on veut rendre grand ou petit. Cependant, les expériences montrent que ces termes ont surtout servi à départager des candidats proches ; le saut de performance vient principalement des familles de graphes initiales.

Partie 2

3 Architecture FunSearch

La Partie 2 implémente une boucle inspirée de FunSearch. Le modèle génère une fonction Python de score, nommée `heuristic_score`. Elle est sauvegardée dans `src/graphbench/funsearch/candidates/`, puis évaluée par le même moteur que la Partie 1. Seul le score change ; les générateurs, mutations, invariants et vérifications restent identiques.

Le cycle est le suivant :

1. lire le registre des candidats déjà évalués ;
2. sélectionner les meilleurs fichiers sources ;
3. construire un prompt avec ces sources et leurs résultats ;
4. appeler OpenRouter pour obtenir une nouvelle fonction Python ;
5. extraire et valider localement la fonction ;
6. lancer l'évaluateur et ajouter une ligne dans `results/funsearch/registry.csv`.

Nous avons d'abord utilisé GPT-5.5 avec raisonnement faible, car il était attendu comme fort pour la génération de code et d'heuristiques, tout en gardant un coût inférieur aux modes de raisonnement plus élevés. Ensuite, pour une séquence plus longue de 25 générations, nous sommes passés à DeepSeek pour des raisons de coût.

Le commit 7527406 correspond à l'expérience GPT-5.5 initiale. Son registre montre six générations principales : `candidate_001` et `candidate_002` trouvent 29/30 sur une évaluation courte, `candidate_003` trouve 28/30, `candidate_004` trouve 97/100, `candidate_005` trouve 98/100, et `candidate_006` ne trouve d'abord que 95/100 dans une ligne du registre. Après réévaluation complète dans `candidate_006_100.csv`, `candidate_006` est le seul candidat de cette série à atteindre 100/100.

3.1 Analyse du candidat GPT-5.5 candidate_006

Le candidat 006 reste proche du score manuel, mais il introduit une idée intéressante : donner un bonus aux graphes proches de la frontière de décision. Il multiplie la violation par 50, puis ajoute un terme gaussien maximal lorsque la violation est proche de zéro.

Listing 7 – Extrait de `candidate_006.py`.

```
score = 50.0 * violation
boundary_weight = math.exp(-5.0 * violation * violation)
diff_ratio = diff_abs / (diff_abs + n + 1.0)
score += 3.0 * boundary_weight * diff_ratio
```

Ce terme est raisonnable pour une recherche locale : un graphe presque contre-exemple est souvent plus utile comme parent qu'un graphe très loin de la frontière. Le candidat ajoute aussi une direction dérivée du polynôme f , utilisée surtout quand la violation est encore négative.

Listing 8 – Guidage par dérivée dans `candidate_006.py`.

```
if sign == '<=':
    deriv_bonus = deriv / (abs(deriv) + 1.0)
else:
    deriv_bonus = -deriv / (abs(deriv) + 1.0)
if violation < 0:
    score += 0.5 * deriv_bonus
```

Enfin, il contient des indices faibles pour les arbres et les graphes sans griffe. Pour les arbres, il favorise les feuilles ; pour les graphes sans griffe, il pénalise légèrement un degré maximal trop élevé.

Ces choix ne suffisent pas à battre la Partie 1, mais ils produisent un score stable et compatible avec le moteur.

3.2 Analyse du candidat DeepSeek candidate_014

Dans la séquence DeepSeek de 25 cycles, `candidate_014` est le meilleur candidat du registre. Il reprend presque la même structure que 006, mais avec une frontière un peu plus accentuée et une pénalité explicite si la direction dérivée va contre l'objectif.

Listing 9 – Extrait de `candidate_014.py`.

```
boundary_weight = math.exp(-6.0 * violation * violation)
diff_ratio = diff_abs / (diff_abs + n + 1.0)
score += 3.5 * boundary_weight * diff_ratio

if sign == '<=':
    direction = deriv
else:
    direction = -deriv
dir_norm = direction / (abs(deriv) + 1.0)
away_penalty = max(0.0, -dir_norm) * 0.3
if violation < 0:
    score += 0.5 * dir_norm - away_penalty
```

Scientifiquement, ce candidat est cohérent : il conserve la violation comme signal principal, encourage les points proches de la frontière et évite certaines directions localement défavorables. Son résultat complet est 100/100, avec un coût total de 166.636643 secondes. Il est donc valide mais plus lent que l'heuristique manuelle et que le candidat GPT-5.5 006.

4 Résultats expérimentaux

Méthode	Trouvé	Coût total	Temps moyen
Heuristique manuelle Partie 1	100/100	136.80	1.368 s
FunSearch GPT-5.5, candidat 006	100/100	157.50	1.575 s
FunSearch DeepSeek, candidat 014	100/100	166.64	1.666 s

TABLE 3 – Résultats finaux sur les 100 conjectures.

La Partie 1 est la meilleure en temps moyen. Elle obtient une médiane de 0.561114 seconde et un maximum de 5.347417 secondes. Le candidat GPT-5.5 006 atteint 100/100 avec un coût total de 157.502736 secondes. Le candidat DeepSeek 014 atteint 100/100 avec un coût total de 166.636643 secondes et constitue le meilleur candidat de la séquence DeepSeek.

Le fait que FunSearch atteigne 100/100 est important : l'architecture produit des fonctions exécutables et valides. En revanche, elle n'améliore pas le temps final. L'explication la plus probable est que le moteur de Partie 1 était déjà très fortement aidé par ses familles initiales. Lorsque le bon graphe est déjà présent dans la population initiale ou proche d'elle, la qualité fine du score devient secondaire.

5 Discussion scientifique

Les conjectures faciles étaient celles pour lesquelles une famille extrémale simple existait : petits graphes complets pour certains cas de clique ou de degré moyen, chemins pour certains cas de distance, araignées pour domination totale contre couplage, doubles étoiles pour domination indépendante et couverture.

Les conjectures difficiles combinaient souvent des invariants coûteux ou sensibles : distance, valeurs propres, domination, domination totale et domination indépendante. Les cas spectraux incluaient notamment la plus grande valeur propre de distance et la deuxième plus petite valeur propre du

Laplacien. La correction de **proximity** et **remoteness** a été un point clé : le benchmark utilise les conventions réciproques

$$\text{proximity} = 1/\max \bar{d}, \quad \text{remoteness} = 1/\min \bar{d}.$$

Avant cette correction, plusieurs cas semblaient difficiles pour de mauvaises raisons.

L'enseignement principal est que la génération de familles de graphes a compté davantage que le façonnage du score. Les mutations locales explorent efficacement autour d'une bonne forme, mais elles découvrent mal certaines structures rares. En ajoutant les bonnes graines, on change la topologie de l'espace de recherche : le moteur commence près des régions où la violation positive existe.

L'IA a été utile pour proposer des fonctions de score, mettre en place la boucle FunSearch et explorer des variantes. Elle a été moins décisive pour identifier les familles extrémales et pour comprendre les conventions spécifiques du benchmark. L'intervention humaine est restée nécessaire pour vérifier les invariants, interpréter les échecs, choisir les graines et maintenir une validation stricte.

5.1 Récapitulatif des questions du sujet

1. Quelles conjectures sont faciles à réfuter ?

Les plus faciles sont celles pour lesquelles une famille extrémale simple existe déjà dans la population initiale (petites cliques, chemins, araignées, doubles étoiles). Dans ces cas, la recherche locale trouve rapidement une violation positive.

2. Quels invariants semblent difficiles à manipuler ?

Les invariants les plus difficiles sont ceux liés aux distances, au spectral et à la domination (`largest_distance_eigenvalue`, `second_smallest_laplace_eigenvalue`, `domination_number`, `total_domination_number`, `independent_domination_number`), car ils sont sensibles à de petites modifications structurelles.

3. Quelles mutations sont efficaces ?

Les mutations locales sont efficaces pour explorer autour d'une bonne forme, mais elles sont surtout performantes quand elles partent de graines structurelles pertinentes. En pratique, leur rôle principal est l'optimisation fine autour de graphes déjà prometteurs.

4. L'architecture FunSearch améliore-t-elle réellement l'heuristique initiale ?

FunSearch améliore l'automatisation de la recherche de scores et produit des candidats valides atteignant 100/100. En revanche, dans nos expériences finales, elle n'améliore pas le temps total par rapport à la meilleure heuristique manuelle de la Partie 1.

5. L'IA a-t-elle produit des idées utiles ou seulement du code ?

L'IA a produit des idées utiles de scoring (guidage proche de la frontière, termes dérivés, ajustements structurels) et une boucle d'évolution exploitable. Cependant, les gains majeurs viennent surtout de la conception humaine des familles initiales et de la validation expérimentale.

6 Dépendances et bibliothèques

La solution est volontairement légère. Le fichier `requirements.txt` contient :

Dépendance	Utilisation
<code>networkx >= 3.2</code>	Représentation des graphes, générateurs, connectivité, line graphs, graph6
<code>numpy >= 1.26</code>	Calcul numérique, notamment pour les invariants spectraux

TABLE 4 – Dépendances Python directes.

La bibliothèque standard Python est utilisée pour les fichiers CSV, les chemins, le temps, l'import dynamique de candidats FunSearch, les nombres aléatoires reproductibles et la gestion JSON. Pour la Partie 2, OpenRouter est appelé via un client local du dépôt ; la clé API n'est pas versionnée. Le rapport est écrit en LaTeX et compilé avec `pdflatex`.